



Laboratory Assessment II

PEC-2 Laboratory: Full Stack Development - BIT26PE05

Run a basic Spring Boot web application

Varad Sushant Deshmukh [123B1F022] TY-IT-A, PCCOE College

Aim

To create and run a basic **Spring Boot web application** using **Spring Initializr (spring.io)** and understand the project structure and execution.

Objectives

- To generate a Spring Boot project using **Spring Initializr**
- To understand the basic project structure (Controller, Service, etc.)

Create Spring Boot app:

Step 1:

Go to: <https://start.spring.io>

This tool auto-generates Spring Boot projects

Step 2: Configure Project

- Project: **Maven**
- Language: **Java**
- Spring Boot: latest stable
- Group: com.example
- Artifact: demo

Step 3: Add Dependencies • Click Add Dependencies

- Select:
 - o **Spring Web**

The screenshot shows the Spring Initializr web application configuration page. The page is titled "spring initializr" and has a green logo. It contains several sections for configuring a Spring Boot project:

- Project:** Radio buttons for "Gradle - Groovy", "Gradle - Kotlin", and "Maven" (selected).
- Language:** Radio buttons for "Java" (selected), "Kotlin", and "Groovy".
- Spring Boot:** Radio buttons for "4.1.0 (SNAPSHOT)", "4.1.0 (M1)", "4.0.0 (SNAPSHOT)", "4.0.3" (selected), "3.0.14 (SNAPSHOT)", and "3.0.13".
- Project Metadata:** Text input fields for "Group" (com.example), "Artifact" (demo), and "Package name" (com.example.demo).
- Packaging:** Radio buttons for "Jar" (selected) and "War".
- Configuration:** Radio buttons for "H2 database" (selected) and "MySQL".
- Dependencies:** A section titled "Spring Web" with a description: "Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container." There is a button "ADD DEPENDENCIES... CTRL + B".
- Buttons:** At the bottom, there are buttons for "GENERATE CTRL + G", "EXPLORE CTRL + SPACE", and a menu icon "...".

Step 4: Generate Project

- Click **Generate**
- ZIP file will download

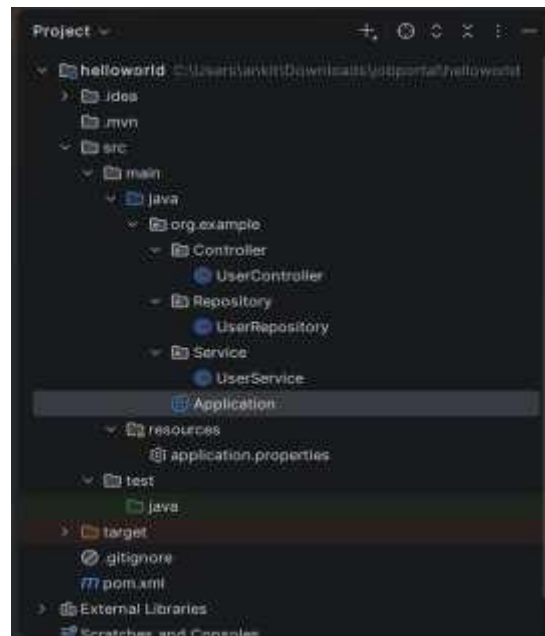
Step 5: Open in IntelliJ

- Extract ZIP
- Open IntelliJ → **Open Project** → select folder

Step 6: Run Application

- Open DemoApplication.java
- Click **Run** ► **Step 7: Test in Browser** Open:

<http://localhost:8080>



Controller Layer

The Controller layer handles HTTP requests and responses. It receives input from the client (browser), calls the Service layer, and returns the output.

Use:

- Define API endpoints (/users, /login)
- Handle request mapping (@GetMapping, @PostMapping)

```

package org.example.Controller; import
org.example.Service.UserService; import
org.springframework.web.bind.annotation.*; import java.util.List;

@RestController

@RequestMapping("/users") public class UserController {
private final UserService userService;  public
UserController(UserService userService) {      this.userService =
userService;

    }

    @GetMapping  public List<String> getUsers() {      return
userService.getUsers();

    }
}

```

Service Layer

The Service layer contains the **business logic** of the application. It processes data, applies rules, and decides what should happen before interacting with the Repository.

Use:

- Process data (validation, calculations)
- Act as a bridge between Controller and Repository

```

package org.example.Service; import
org.example.Repository.UserRepository; import
org.springframework.stereotype.Service; import java.util.List;

@Service public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
this.userRepository = userRepository;

    }

    public List<String> getUsers() {      return
userRepository.getAllUsers();

    }
}

```

Repository Layer

The Repository layer handles **data access operations**. It communicates with the database to store, retrieve, update, or delete data.

Use:

- Perform CRUD operations
- Interact with database (via JPA or other methods)
- **Flow (Important)**
- Controller → Service → Repository → Database

```
package org.example.Repository; import org.springframework.stereotype.Repository; import java.util.*; @Repository
public class UserRepository {
```

```
    private List<String> users = new ArrayList<>();
    public UserRepository() {    users.add("John");
    users.add("Alice");

    }

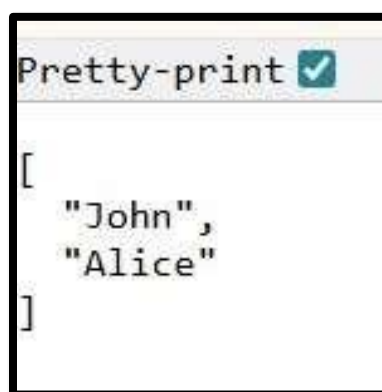
    public List<String> getAllUsers() {    return users;

    }

}
```

Output:

Hit: <http://localhost:8081/users>



Conclusion:

The assignment successfully demonstrated the creation of a Spring Boot application using Spring Initializr, which simplifies project setup by automatically configuring dependencies and structure. The application was executed and tested successfully, providing a clear understanding of how Spring Boot applications are generated and run.